

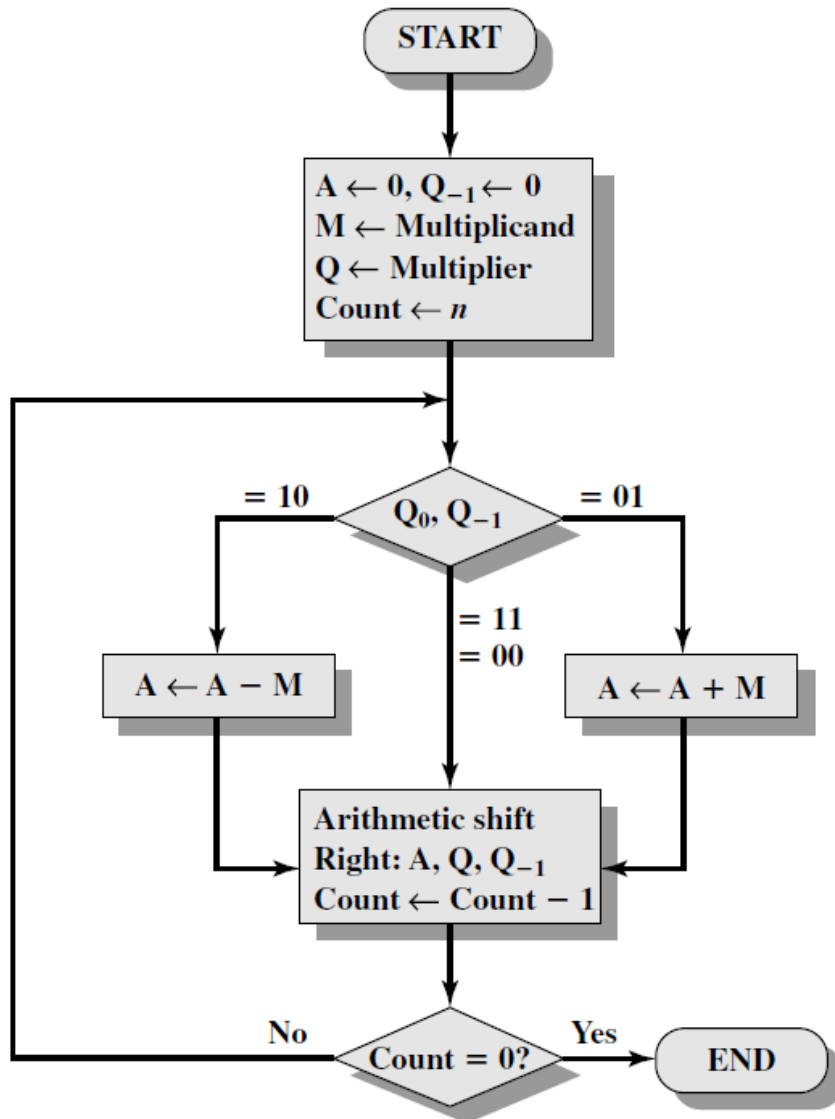
BECE102L - Digital Systems Design

Dr Arunachalam V

Associate Professor, SENSE

Booth's Multiplier – examples & Verilog model

Booth's Algorithm for 2's Complement Multiplication



A	Q	Q ₋₁	M		
0000	0011	0	0111	Initial values	
1001	0011	0	0111	A ← A - M } Shift	First cycle
1100	1001	1	0111		
1110	0100	1	0111	Shift }	Second cycle
0101	0100	1	0111	A ← A + M } Shift	Third cycle
0010	1010	0	0111		
0001	0101	0	0111	Shift }	Fourth cycle

- In either case, the right shift is such that the leftmost bit of A, namely, A_{n-1} not only is shifted into A_{n-2} but also remains in A_{n-1} .
- This is required to preserve the sign of the number in A and Q.
- It is known as an **arithmetic shift**, because it preserves the sign bit.

M(7)×Q (-3)

A	Q (-3)	Q ₋₁	M (7)	C	Remarks
0000	1101	0	0111	4	2's (0011) = 1101
1001	1101	0	0111	3	(10), $A \leftarrow A - M$; A = 0000 + 1001 = 1001; 2's (0111) = 1001
1100	1110	1	0111		Shift right (Arithmetic)
0011	1110	1	0111	2	(01), $A \leftarrow A + M$; A = 1100 + 0111 = 0011;
0001	1111	0	0111		Shift right (Arithmetic)
1010	1111	0	0111	1	(10), $A \leftarrow A - M$; A = 0001 + 1001 = 1010; 2's (0111) = 1001
1101	0111	1	0111		Shift right (Arithmetic)
1110	1011	1	0111	0	(11), Shift right (Arithmetic)

M(-7)×Q (3)

A	Q (3)	Q ₋₁	M (-7)	C	Remarks
0000	0011	0	1001	4	2's (0111) = 1101
0111	0011	0	1001	3	(10), $A \leftarrow A - M$; $A = 0000 + 0111 = 0111$; 2's (1001)=0111
0011	1001	1	1001		Shift right (Arithmetic)
0001	1100	1	1001	2	(11), Shift right (Arithmetic)
1010	1100	1	1001	1	(01), $A \leftarrow A + M$; $A = 0001 + 1001 = 1010$; 2's (0111)=1001
1101	0110	0	1001		Shift right (Arithmetic)
1110	1011	1	1001	0	(00), Shift right (Arithmetic)

$$M(-7) \times Q(-3)$$

A	$Q(-3)$	Q_{-1}	M (-7)	C	Remarks
0000	1101	0	1001	4	2's (0011) = 1101; 2's (0111) = 1001;
0111	1101	0	1001	3	(10), $A \leftarrow A - M$; $A = 0000 + 0111 = 0111$; 2's (1001) = 0111
0011	1110	1	1001		Shift right (Arithmetic)
1100	1110	1	1001	2	(01), $A \leftarrow A + M$; $A = 0011 + 1001 = 1100$;
1110	0111	0	1001		(11), Shift right (Arithmetic)
0101	0111	0	1001	1	(10), $A \leftarrow A - M$; $A = 1110 + 0111 = 1010$; 2's (1001) = 0111
0010	1011	1	1001		Shift right (Arithmetic)
0001	0101	1	1001	0	(11), Shift right (Arithmetic)

Verilog model (combinational logic) – Booth multiplier (4-bit)

```
module booth_multiplier_4bit (  
input [3:0] multiplicand,  
input [3:0] multiplier,  
output [7:0] product );  
  
// Internal registers for Booth's algorithm  
reg [3:0] A; // Accumulator  
reg [3:0] Q; // Multiplier register  
reg [3:0] M; // Multiplicand register  
reg Qm1; // Q-1 bit for Booth encoding  
  
// Result register  
reg [7:0] result;  
  
// Loop variable for iterations  
  
integer i;  
  
// Combinational logic for Booth's algorithm  
  
always @* begin  
  
// Initialize registers  
A = 4'b0000;  
Q = multiplier;  
M = multiplicand;  
Qm1 = 1'b0;  
result = 8'b00000000; // Initialize result
```

Verilog model (combinational logic) – Booth multiplier (4-bit)

// Iterate through the Booth's algorithm steps

*for (i = 0; i < 4; i = i + 1) begin
case ({Q[0], Qm1}) // Examine the last two bits of
the multiplier*

2'b01: A = A + M; // Add multiplicand if 01

2'b10: A = A - M; // Subtract multiplicand if 10

default: ; // No operation for 00 or 11

endcase

// Arithmetic right shift of {A, Q, Qm1}

*{A, Q, Qm1} = {A[3], A, Q, Qm1} >> 1;
end*

// Final product is the concatenation of A and Q

*result = {A, Q};
end*

// Assign the internal result to the output

assign product = result;

endmodule


```

module booth_multiplier_4bit (
input [3:0] multiplicand,
input [3:0] multiplier,
output [7:0] product );
reg [3:0] A; // Accumulator
reg [3:0] Q; // Multiplier register
reg [3:0] M; // Multiplicand register
reg Qm1; // Q-1 bit for Booth encoding
reg [7:0] result; // Result register
integer i; // Loop variable for iterations
always @* begin
A = 4'b0000; // Initialize registers
Q = multiplier;
M = multiplicand;
Qm1 = 1'b0;
result = 8'b00000000; // Initialize result

```

```

// Iterate through the Booth's algorithm steps
for (i = 0; i < 4; i = i + 1) begin
case ({Q[0], Qm1}) // Examine (Q0,Q-1)bits
2'b01: A = A + M; // Add multiplicand if 01
2'b10: A = A - M; // Subtract multiplicand if 10
default: ; // No operation for 00 or 11
endcase
{A, Q, Qm1} = {A[3], A, Q, Qm1} >> 1; // Arithmetic right
shift
end

result = {A, Q}; // Final product, concatenation of A & Q
end
assign product = result; // Assign result to the output
endmodule

```

Testbench – Booth multiplier (4-bit)

```
`timescale 1ns / 1ps
module booth_multiplier_tb;
// Declare signals for inputs and outputs of the Booth multiplier module
reg [3:0] multiplicand; // Example: 4-bit multiplicand
reg [3:0] multiplier; // Example: 4-bit multiplier
wire [7:0] product; // Example: 8-bit product for 4x4 multiplication
// Instantiate the Booth multiplier module (Unit Under Test - UUT)
booth_multiplier uut (
    .multiplicand_in(multiplicand),
    .multiplier_in(multiplier),
    .product_out(product)
);
// Initial block for stimulus generation
initial begin
// Initialize inputs
    multiplicand = 4'd0; multiplier = 4'd0;
```

```

// Apply test cases
    #10; // Wait for a small delay
    multiplicand = 4'd5; multiplier = 4'd3;
    #10; $display("Test Case 1: %d * %d = %d", multiplicand, multiplier, product);
    #10; multiplicand = 4'd7; multiplier = 4'd-3;
    #10; $display("Test Case 2: %d * %d = %d", multiplicand, multiplier, product);
    #10; multiplicand = 4'd-5; multiplier = 4'd4; // Signed number example (two's complement)
    #10; $display("Test Case 3: %d * %d = %d", $signed(multiplicand), $signed(multiplier), $signed(product));
    #10; multiplicand = 4'd-6; multiplier = 4'd-2;
    #10; $display("Test Case 4: %d * %d = %d", $signed(multiplicand), $signed(multiplier), $signed(product));
// Add more test cases as needed for thorough verification
    #10; // Wait for final result display
    $finish; // Terminate simulation
end
// Optional: Monitor and display signals during simulation
initial begin
    $monitor("Time = %0t, Multiplicand = %d, Multiplier = %d, Product = %d",
        $time, $signed(multiplicand), $signed(multiplier), $signed(product));
end
endmodule

```

Next class

Magnitude comparator and its Verilog modelling